Contents

Lab Manual — GUI Components, Fonts, and Colors

Subject: Mobile Application Development (MAD)
Project: GUIComponentsFontsAndColors
Package: `com.example.guicomponentsfontsandcolors`
Technologies: Java, XML, AndroidX Navigation, Material Design, View Binding

---

Table of Contents

---

1. Introduction

1.1 Purpose

This lab manual documents the Android application **GUI Components, Fonts, and Colors**, developed as part of the MAD curriculum. The app demonstrates:

- Common **GUI widgets** (`TextView`, `Button`, `EditText`, `CheckBox`, `RadioButton`, `Switch`, `ProgressBar`)
- **Fonts** (`typeface`: sans, serif, monospace; size and style)
- **Colors** (resource-defined colors and `backgroundTint`)
- **Layouts** (`LinearLayout`, `ScrollView`, `ConstraintLayout`, `CoordinatorLayout`)
- **Fragments** and **Navigation Component**
- **View Binding** connecting XML layouts to Java code

1.2 Learning Outcomes

After completing this lab, students will be able to:

1. Explain the standard Android project folder structure.
2. Trace application execution from manifest launch to fragment navigation.
3. Define colors, strings, themes, and dimensions in `res/values/`.
4. Design screens using XML layout files.
5. Implement `Activity` and `Fragment` classes with View Binding.
6. Configure Navigation Component using `nav_graph.xml`.

1.3 App Overview

Item	Value
App name	GUI Components , Fonts, and Colors.
Launcher activity	`MainActivity`
Start screen	`FirstFragment` (GUI demo)
Second screen	`SecondFragment` (ConstraintLayout + long text)
Min SDK	24
Target SDK	36
Language	Java 11

---

2. Project Structure (Structural View)

2.1 Root Project Tree

```
GUIComponentsFontsandColors/
├── app/                          ← Application module
│   ├── build.gradle              ← Module build & dependencies
│   └── src/
│       ├── main/                 ← Production code & resources
│       │   ├── AndroidManifest.xml
│       │   ├── java/com/example/guicomponentsfontsandcolors/
│       │   │   ├── MainActivity.java
│       │   │   ├── FirstFragment.java
│       │   │   └── SecondFragment.java
│       │   └── res/
│       │       ├── layout/       ← Screen layouts
│       │       ├── navigation/   ← Nav graph
│       │       ├── menu/         ← Options menu
│       │       ├── values/       ← colors, strings, themes, dimens
│       │       ├── values-night/
│       │       ├── values-v23/
│       │       ├── values-land/
│       │       ├── values-w600dp/
│       │       ├── values-w1240dp/
│       │       ├── drawable/
│       │       ├── mipmap-*/
│       │       └── xml/
│       ├── test/                 ← Unit tests
│       └── androidTest/          ← Instrumented tests
├── build.gradle                  ← Project-level Gradle
├── gradle/
│   └── libs.versions.toml        ← Version catalog
└── docs/
    └── LAB_MANUAL.md             ← This document
```

2.2 Main Module — Logical Layers

```
Presentation Layer
• activity_main.xml, content_main.xml
• fragment_first.xml, fragment_second.xml
• menu_main.xml

Controller Layer (Java)
• MainActivity.java
• FirstFragment.java, SecondFragment.java

Navigation Layer
• nav_graph.xml

Resource Layer
• values/: colors, strings, themes, dimens

Configuration Layer
• AndroidManifest.xml, build.gradle
```

2.3 Key Files at a Glance

Category	Files
Entry	`AndroidManifest.xml`
Activity shell	`activity_main.xml`, `content_main.xml`, `MainActivity.java`
GUI lab screen	`fragment_first.xml`, `FirstFragment.java`
Navigation demo	`fragment_second.xml`, `SecondFragment.java`, `nav_graph.xml`
Resources	`colors.xml`, `strings.xml`, `themes.xml`, `dimens.xml`

---

3. Application Flow (Top to Bottom)

Execution order when the user launches the app:

```
STEP 1  User taps app icon
            |
STEP 2  Android reads AndroidManifest.xml
            |  → Application theme applied
            |  → MainActivity started (LAUNCHER intent)
            |
STEP 3  MainActivity.onCreate() runs
            |  → EdgeToEdge enabled
            |  → activity_main.xml inflated (View Binding)
            |  → Toolbar set as ActionBar
            |  → NavHostFragment found in content_main.xml
            |  → Navigation linked to ActionBar (Up button)
            |  → FAB click → Snackbar
            |
STEP 4  NavHostFragment loads nav_graph.xml
```

```
            |  → startDestination = FirstFragment
            |
STEP 5  FirstFragment.onCreateView()
            |  → fragment_first.xml inflated
            |  → GUI demo visible (fonts, colors, widgets)
            |
STEP 6  FirstFragment.onViewCreated()
            |  → button_first click listener registered
            |
STEP 7  [User taps "Go to Second Fragment"]
            |  → NavController.navigate(action_FirstFragment_to_SecondFragment)
            |
STEP 8  SecondFragment displayed
            |  → fragment_second.xml (Previous + lorem ipsum)
            |  → ActionBar shows "Second Fragment", Up enabled
            |
STEP 9  [User taps "Previous" OR ActionBar Up]
            |  → Back to FirstFragment
            |
STEP 10 [Any time] Overflow menu → Settings (handled in MainActivity)
        [Any time] FAB → Snackbar
```

3.1 Screen Flow Diagram

```
flowchart TD
    A[App Launch] --> B[MainActivity]
    B --> C[activity_main + content_main]
    C --> D[NavHost + nav_graph]
    D --> E[FirstFragment<br/>GUI / Fonts / Colors]
    E -->|button_first| F[SecondFragment]
    F -->|button_second or Up| E
    B --> G[FAB → Snackbar]
    B --> H[Menu → Settings]
```

---

4. Configuration Files

Configuration defines **how the app is built** and **how Android starts it**.

4.1 AndroidManifest.xml

Path: app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GUIComponentsFontsAndColors">
```

```
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.GUIComponentsFontsAndColors">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Element	Role
`android:theme`	Applies Material3 DayNight theme app-wide
`android:name=".MainActivity"`	Launcher activity class
`MAIN + LAUNCHER`	Shows app icon in launcher
`android:exported="true"`	Required for launcher on modern Android

4.2 App Module — build.gradle

Path: `app/build.gradle`

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace 'com.example.guicomponentsfontsandcolors'
    compileSdk {
        version = release(36) {
            minorApiLevel = 1
        }
    }

    defaultConfig {
        applicationId "com.example.guicomponentsfontsandcolors"
        minSdk 24
        targetSdk 36
        versionCode 1
        versionName "1.0"

        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-
        }
    }
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_11
```

```
        targetCompatibility JavaVersion.VERSION_11
    }
    buildFeatures {
        viewBinding true
    }
}

dependencies {
    implementation libs.activity.ktx
    implementation libs.appcompat
    implementation libs.constraintlayout
    implementation libs.material
    implementation libs.navigation.fragment
    implementation libs.navigation.ui
    testImplementation libs.junit
    androidTestImplementation libs.espresso.core
    androidTestImplementation libs.ext.junit
}
```

Setting	Purpose
`viewBinding true`	Generates *Binding classes for layouts
`minSdk 24`	Android 7.0+
Navigation & Material libs	Fragments navigation, Toolbar, FAB, Switch

---

4.3 Project build.gradle

Path: `build.gradle`

```
// Top-level build file where you can add configuration options common to all sub-projects/mo
plugins {
    alias(libs.plugins.android.application) apply false
}
```

4.4 Navigation Graph (Configuration)

Path: `app/src/main/res/navigation/nav_graph.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/FirstFragment">

    <fragment
        android:id="@+id/FirstFragment"
        android:name="com.example.guicomponentsfontsandcolors.FirstFragment"
        android:label="@string/first_fragment_label"
        tools:layout="@layout/fragment_first">

        <action
            android:id="@+id/action_FirstFragment_to_SecondFragment"
            app:destination="@id/SecondFragment" />
```

```
    </fragment>
    <fragment
        android:id="@+id/SecondFragment"
        android:name="com.example.guicomponentsfontsandcolors.SecondFragment"
        android:label="@string/second_fragment_label"
        tools:layout="@layout/fragment_second">

        <action
            android:id="@+id/action_SecondFragment_to_FirstFragment"
            app:destination="@id/FirstFragment" />
    </fragment>
</navigation>
```

4.5 Menu Configuration

Path: `app/src/main/res/menu/menu_main.xml`

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context="com.example.guicomponentsfontsandcolors.MainActivity">
    <item
        android:id="@+id/action_settings"
        android:orderInCategory="100"
        android:title="@string/action_settings"
        app:showAsAction="never" />
</menu>
```

4.6 Backup / Data Rules (Reference)

Path: `app/src/main/res/xml/backup_rules.xml` — used by `android:fullBackupContent` in manifest.

---

5. Resource Files (XML — values/)

Resources in `res/values/` are compiled into the R class and referenced as `@string/...`, `@color/...`, etc.

5.1 colors.xml

Path: `app/src/main/res/values/colors.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="purple_200">#FFBB86FC</color>
    <color name="purple_500">#FF6200EE</color>
    <color name="purple_700">#FF3700B3</color>
    <color name="teal_200">#FF03DAC5</color>
    <color name="teal_700">#FF018786</color>
    <color name="red">#FF0000</color>
    <color name="green">#00FF00</color>
```

```
    <color name="blue">#0000FF</color>
    <color name="yellow">#FFFF00</color>
    <color name="orange">#FFA500</color>
</resources>
```

Used in app: purple_500 (title text), red / green / blue (button tints).

5.2 strings.xml

Path: app/src/main/res/values/strings.xml

```
<resources>
    <string name="app_name">GUI Components , Fonts,  and Colors.</string>
    <string name="action_settings">Settings</string>
    <!-- Strings used for fragments for navigation -->
    <string name="first_fragment_label">First Fragment</string>
    <string name="second_fragment_label">Second Fragment</string>
    <string name="next">Next</string>
    <string name="previous">Previous</string>

    <string name="lorem_ipsum">
        Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nam in scelerisque sem. Maur
        volutpat, dolor id interdum ullamcorper, risus dolor egestas lectus, sit amet mattis
        dui nec risus. Maecenas non sodales nisi, vel dictum dolor. Class aptent taciti socio
        litora torquent per conubia nostra, per inceptos himenaeos. Suspendisse blandit eleif
        diam, vel rutrum tellus vulputate quis. Aliquam eget libero aliquet, imperdiet nisl a
        ornare ex. Sed rhoncus est ut libero porta lobortis. Fusce in dictum tellus.\n\n
        Suspendisse interdum ornare ante. Aliquam nec cursus lorem. Morbi id magna felis. Viv
        egestas, est a condimentum egestas, turpis nisl iaculis ipsum, in dictum tellus dolor
        neque. Morbi tellus erat, dapibus ut sem a, iaculis tincidunt dui. Interdum et malesu
        fames ac ante ipsum primis in faucibus. Curabitur et eros porttitor, ultricies urna v
        molestie nibh. Phasellus at commodo eros, non aliquet metus. Sed maximus nisl nec dol
        bibendum, vel congue leo egestas.\n\n
        Sed interdum tortor nibh, in sagittis risus mollis quis. Curabitur mi odio, condiment
        amet auctor at, mollis non turpis. Nullam pretium libero vestibulum, finibus orci vel
        molestie quam. Fusce blandit tincidunt nulla, quis sollicitudin libero facilisis et.
        interdum nunc ligula, et fermentum metus hendrerit id. Vestibulum lectus felis, dictu
        lacinia sit amet, tristique id quam. Cras eu consequat dui. Suspendisse sodales nunc
        in lobortis sem porta sed. Integer id ultrices magna, in luctus elit. Sed a pellentes
        est.\n\n
        Aenean nunc velit, lacinia sed dolor sed, ultrices viverra nulla. Etiam a venenatis n
        Morbi laoreet, tortor sed facilisis varius, nibh orci rhoncus nulla, id elementum leo
        non lorem. Nam mollis ipsum quis auctor varius. Quisque elementum eu libero sed commo
        eros nisl, imperdiet vel imperdiet et, scelerisque a mauris. Pellentesque varius ex n
        quis imperdiet eros placerat ac. Duis finibus orci et est auctor tincidunt. Sed non v
        ipsum. Nunc quis augue egestas, cursus lorem at, molestie sem. Morbi a consectetur ip
        placerat diam. Etiam vulputate dignissim convallis. Integer faucibus mauris sit amet
        convallis.\n\n
        Phasellus in aliquet mi. Pellentesque habitant morbi tristique senectus et netus et
        malesuada fames ac turpis egestas. In volutpat arcu ut felis sagittis, in finibus mas
        gravida. Pellentesque id tellus orci. Integer dictum, lorem sed efficitur ullamcorper
        libero justo consectetur ipsum, in mollis nisl ex sed nisl. Donec maximus ullamcorper
        sodales. Praesent bibendum rhoncus tellus nec feugiat. In a ornare nulla. Donec rhonc
        libero vel nunc consequat, quis tincidunt nisl eleifend. Cras bibendum enim a justo l
        vestibulum. Fusce dictum libero quis erat maximus, vitae volutpat diam dignissim.
    </string>
```

```
</resources>
```

5.3 themes.xml (Light)

Path: app/src/main/res/values/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.GUIComponentsFontsAndColors" parent="Theme.Material3.DayNight.NoA
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>

    <style name="Theme.GUIComponentsFontsAndColors" parent="Base.Theme.GUIComponentsFontsAndC
</resources>
```

5.4 themes.xml (Night)

Path: app/src/main/res/values-night/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.GUIComponentsFontsAndColors" parent="Theme.Material3.DayNight.NoA
        <!-- Customize your dark theme here. -->
        <!-- <item name="colorPrimary">@color/my_dark_primary</item> -->
    </style>
</resources>
```

5.5 themes.xml (API 23+ — Edge to Edge)

Path: app/src/main/res/values-v23/themes.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Theme.GUIComponentsFontsAndColors" parent="Base.Theme.GUIComponentsFontsAndC
        <!-- Transparent system bars for edge-to-edge. -->
        <item name="android:navigationBarColor">@android:color/transparent</item>
        <item name="android:statusBarColor">@android:color/transparent</item>
        <item name="android:windowLightStatusBar">?attr/isLightTheme</item>
    </style>
</resources>
```

5.6 dimens.xml

Default — app/src/main/res/values/dimens.xml

```
<resources>
    <dimen name="fab_margin">16dp</dimen>
</resources>
```

Landscape override — app/src/main/res/values-land/dimens.xml

```xml
<resources>
    <dimen name="fab_margin">48dp</dimen>
</resources>
```

Note: Wider margins in landscape for FAB positioning.

5.7 Resource Reference Table

Resource	File	Referenced by
`@string/app_name`	strings.xml	Manifest, launcher label
`@color/red, green, blue`	colors.xml	fragment_first.xml buttons
`@color/purple_500`	colors.xml	fragment_first.xml title
`@string/lorem_ipsum`	strings.xml	fragment_second.xml
`@string/previous`	strings.xml	fragment_second.xml button
`@style/Theme.GUIComponentsFragmentsAndXml`	themes.xml	Manifest
`@dimen/fab_margin`	dimens.xml	activity_main.xml FAB

---

6. Layout Files (XML — res/layout/)

Layouts define **what the user sees**. They are inflated by Activities and Fragments.

6.1 activity_main.xml — Activity Shell

Path: `app/src/main/res/layout/activity_main.xml`
Inflated by: `MainActivity` → `ActivityMainBinding`

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.co
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fitsSystemWindows="true"
    tools:context=".MainActivity">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:fitsSystemWindows="true">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize" />

    </com.google.android.material.appbar.AppBarLayout>

    <include layout="@layout/content_main" />

    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fab"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|end"
        android:layout_marginEnd="@dimen/fab_margin"
        android:layout_marginBottom="16dp"
        app:srcCompat="@android:drawable/ic_dialog_email" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Component	Purpose
`CoordinatorLayout`	Coordinates AppBar, content, FAB
`MaterialToolbar`	Top app bar (ActionBar)
`<include content_main>`	Hosts NavHostFragment
`FloatingActionButton`	Triggers Snackbar in Java

6.2 content_main.xml — Fragment Container

Path: `app/src/main/res/layout/content_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:layout_behavior="@string/appbar_scrolling_view_behavior">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment_content_main"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:defaultNavHost="true"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        app:navGraph="@navigation/nav_graph" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Attribute	Purpose
`app:navGraph`	Loads `nav_graph.xml`
`app:defaultNavHost="true"`	System Back delegates to NavController

6.3 fragment_first.xml — GUI / Fonts / Colors Demo

Path: `app/src/main/res/layout/fragment_first.xml`
Inflated by: `FirstFragment` → `FragmentFirstBinding`

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".FirstFragment">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="GUI Components Demo"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="@color/purple_500"
        android:layout_gravity="center_horizontal"
        android:layout_marginBottom="24dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Fonts Demonstration:"
        android:textSize="18sp"
        android:layout_marginBottom="8dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is Sans-Serif font (Default)"
        android:typeface="sans"
        android:layout_marginBottom="4dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is Serif font"
        android:typeface="serif"
        android:layout_marginBottom="4dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is Monospace font"
        android:typeface="monospace"
        android:layout_marginBottom="16dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Colors and Buttons:"
        android:textSize="18sp"
        android:layout_marginBottom="8dp" />

    <Button
```

```
    android:id="@+id/button_red"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Red Button"
    android:backgroundTint="@color/red"
    android:layout_marginBottom="8dp" />

<Button
    android:id="@+id/button_green"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Green Button"
    android:backgroundTint="@color/green"
    android:layout_marginBottom="8dp" />

<Button
    android:id="@+id/button_blue"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Blue Button"
    android:backgroundTint="@color/blue"
    android:layout_marginBottom="16dp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Other Components:"
    android:textSize="18sp"
    android:layout_marginBottom="8dp" />

<EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Type something here..."
    android:layout_marginBottom="8dp" />

<CheckBox
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Check me"
    android:layout_marginBottom="8dp" />

<RadioGroup
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginBottom="8dp">
    <RadioButton
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Option 1" />
    <RadioButton
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Option 2" />
</RadioGroup>
```

```
        <com.google.android.material.materialswitch.MaterialSwitch
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Enable something"
            android:layout_marginBottom="16dp" />

        <ProgressBar
            style="?android:attr/progressBarStyleHorizontal"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:progress="50"
            android:layout_marginBottom="24dp" />

        <Button
            android:id="@+id/button_first"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Go to Second Fragment" />

    </LinearLayout>
</ScrollView>
```

Section summary:

Lines (section)	Topic
Title TextView	Size, bold, color
Three TextViews	typeface: sans, serif, monospace
Three Buttons	backgroundTint with @color/
EditText, CheckBox, RadioGroup, Switch, ProgressBar	Common GUI components
button_first	Navigation trigger (wired in Java)

6.4 fragment_second.xml — Second Screen

Path: app/src/main/res/layout/fragment_second.xml
Inflated by: SecondFragment → FragmentSecondBinding

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.core.widget.NestedScrollView xmlns:android="http://schemas.android.com/apk/res/andr
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".SecondFragment">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="16dp">

        <Button
            android:id="@+id/button_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
            android:text="@string/previous"
            app:layout_constraintBottom_toTopOf="@id/textview_second"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />

        <TextView
            android:id="@+id/textview_second"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="@string/lorem_ipsum"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toBottomOf="@id/button_second" />
    </androidx.constraintlayout.widget.ConstraintLayout>
</androidx.core.widget.NestedScrollView>
```

Compare layouts:

Feature	fragment_first	fragment_second
Root	ScrollView	NestedScrollView
Child layout	LinearLayout (vertical)	ConstraintLayout
Focus	GUI teaching	Navigation + constraints

---

7. Java Source Files (Top-to-Bottom Flow)

Java runs **after** manifest and resources are ready. Order matches runtime.

7.1 MainActivity.java — Application Host

Path: app/src/main/java/com/example/guicomponentsfontsandcolors/MainActivity.java
Runs: First, on every cold start

```java
package com.example.guicomponentsfontsandcolors;

import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import androidx.navigation.NavController;
import androidx.navigation.fragment.NavHostFragment;
import androidx.navigation.ui.AppBarConfiguration;
import androidx.navigation.ui.NavigationUI;

import com.example.guicomponentsfontsandcolors.databinding.ActivityMainBinding;
```

```java
import com.google.android.material.snackbar.Snackbar;

public class MainActivity extends AppCompatActivity {

    private AppBarConfiguration appBarConfiguration;
    private ActivityMainBinding binding;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);

        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        ViewCompat.setOnApplyWindowInsetsListener(binding.main, (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom
            return insets;
        });
        setSupportActionBar(binding.toolbar);

        NavHostFragment navHostFragment = (NavHostFragment) getSupportFragmentManager().findF
        if (navHostFragment != null) {
            NavController navController = navHostFragment.getNavController();
            appBarConfiguration = new AppBarConfiguration.Builder(navController.getGraph()).b
            NavigationUI.setupActionBarWithNavController(this, navController, appBarConfigura
        }

        binding.fab.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Snackbar.make(view, "Replace with your own action", Snackbar.LENGTH_LONG)
                        .setAnchorView(R.id.fab)
                        .setAction("Action", null).show();
            }
        });
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        // Handle action bar item clicks here. The action bar will
        // automatically handle clicks on the Home/Up button, so long
        // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();

        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }
```

```
        return super.onOptionsItemSelected(item);
    }

    @Override
    public boolean onSupportNavigateUp() {
        NavHostFragment navHostFragment = (NavHostFragment) getSupportFragmentManager().findF
        if (navHostFragment != null) {
            NavController navController = navHostFragment.getNavController();
            return NavigationUI.navigateUp(navController, appBarConfiguration)
                    || super.onSupportNavigateUp();
        }
        return super.onSupportNavigateUp();
    }
}
```

Method execution order in `onCreate`:

1. `super.onCreate`
2. `EdgeToEdge.enable`
3. Inflate `activity_main` via binding
4. `setContentView`
5. Window insets padding
6. `setSupportActionBar(toolbar)`
7. Setup `NavController` + ActionBar
8. FAB listener

7.2 FirstFragment.java — Start Destination

Path: `app/src/main/java/com/example/guicomponentsfontsandcolors/FirstFragment.java`
Runs: After NavHost loads graph (automatically)

```
package com.example.guicomponentsfontsandcolors;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;

import com.example.guicomponentsfontsandcolors.databinding.FragmentFirstBinding;

public class FirstFragment extends Fragment {

    private FragmentFirstBinding binding;

    @Override
    public View onCreateView(
            @NonNull LayoutInflater inflater, ViewGroup container,
            Bundle savedInstanceState
    ) {

        binding = FragmentFirstBinding.inflate(inflater, container, false);
        return binding.getRoot();
```

```
    }

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        binding.buttonFirst.setOnClickListener(v ->
                NavHostFragment.findNavController(FirstFragment.this)
                        .navigate(R.id.action_FirstFragment_to_SecondFragment)
        );
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }

}
```

Callback	Action
onCreateView	Inflate fragment_first.xml
onViewCreated	Navigate to Second on button_first click
onDestroyView	Release binding reference

7.3 SecondFragment.java — Destination Screen

Path: app/src/main/java/com/example/guicomponentsfontsandcolors/SecondFragment.java
Runs: After navigation from FirstFragment

```
package com.example.guicomponentsfontsandcolors;

import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.fragment.app.Fragment;
import androidx.navigation.fragment.NavHostFragment;

import com.example.guicomponentsfontsandcolors.databinding.FragmentSecondBinding;

public class SecondFragment extends Fragment {

    private FragmentSecondBinding binding;

    @Override
    public View onCreateView(
            @NonNull LayoutInflater inflater, ViewGroup container,
            Bundle savedInstanceState
    ) {

        binding = FragmentSecondBinding.inflate(inflater, container, false);
```

```
        return binding.getRoot();

    }

    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);

        binding.buttonSecond.setOnClickListener(v ->
                NavHostFragment.findNavController(SecondFragment.this)
                        .navigate(R.id.action_SecondFragment_to_FirstFragment)
        );
    }

    @Override
    public void onDestroyView() {
        super.onDestroyView();
        binding = null;
    }

}
```

---

8. Complete Lifecycle Flow (Summary)

8.1 Activity Lifecycle (MainActivity)

```
onCreate  → UI visible, NavHost starts FirstFragment
onStart   → Activity visible
onResume  → Interactive
[User interacts]
onPause   → Partially obscured (if applicable)
onStop    → Not visible
onDestroy → Activity destroyed
```

8.2 Fragment Lifecycle (First / Second)

```
onAttach → onCreate → onCreateView → onViewCreated → STARTED → RESUMED
         [User navigates away]
onPause → onStop → onDestroyView (binding = null) → onDestroy → onDetach
```

8.3 End-to-End Timeline (Single Session)

#	Event	Active component
1	Tap launcher icon	System
2	Parse Manifest	`AndroidManifest.xml`
3	Apply theme	`themes.xml`
4	`MainActivity.onCreate`	Java + `activity_main.xml`
5	Inflate `content_main`	`NavHostFragment`
6	Load `nav_graph`	`FirstFragment`
7	Show GUI demo	`fragment_first.xml`
8	Tap FAB	Snackbar
9	Tap overflow → Settings	`menu_main.xml` handler
10	Tap “Go to Second Fragment”	`FirstFragment` → NavController
11	Show second screen	`fragment_second.xml`
12	Tap Previous or Up	`SecondFragment` or `MainActivity`

#	Event	Active component
13	Back on First	FirstFragment recreated

---

9. File Relationship Diagram

9.1 Dependency Graph (ASCII)

```
AndroidManifest.xml
    ├── @style/Theme... ──────────────► themes.xml (+ values-night, values-v23)
    ├── @string/app_name ─────────────► strings.xml
    └── starts MainActivity.java
            │
            ├── ActivityMainBinding ◄── activity_main.xml
            │       ├── @layout/content_main ──► content_main.xml
            │       │       └── app:navGraph ──► nav_graph.xml
            │       │                   ├── FirstFragment.java ◄── fragment_first.xml
            │       │                   │       └── @color/* ──► colors.xml
            │       │                   └── SecondFragment.java ◄── fragment_second.xml
            │       │                           └── @string/lorem_ipsum ──► strings.xml
            │       └── @dimen/fab_margin ──► dimens.xml
            │
            ├── R.menu.menu_main ─────────► menu_main.xml
            └── NavController / AppBar ◄──── nav_graph.xml
```

9.2 Mermaid — File Relationships

```
graph LR
    subgraph Config
        M[AndroidManifest.xml]
        G[build.gradle]
        N[nav_graph.xml]
    end
    subgraph Values
        C[colors.xml]
        S[strings.xml]
        T[themes.xml]
        D[dimens.xml]
    end
    subgraph Layouts
        AM[activity_main.xml]
        CM[content_main.xml]
        F1[fragment_first.xml]
        F2[fragment_second.xml]
    end
    subgraph Java
        MA[MainActivity.java]
        FF[FirstFragment.java]
        SF[SecondFragment.java]
    end
    M --> MA
    T --> M
    S --> M
    G --> MA
    MA --> AM
```

```
AM --> CM
CM --> N
N --> FF
N --> SF
FF --> F1
SF --> F2
F1 --> C
F2 --> S
AM --> D
MA --> S
```

9.3 View Binding Map

Layout file	Generated binding class	Used in
`activity_main.xml`	`ActivityMainBinding`	`MainActivity`
`fragment_first.xml`	`FragmentFirstBinding`	`FirstFragment`
`fragment_second.xml`	`FragmentSecondBinding`	`SecondFragment`

---

10. Lab Exercises for Students

Lab 1 — Project Familiarization

1. Import the project and run it on an emulator (API 24+).
2. Draw the folder tree from Section 2 on paper and label each file's role.
3. **Submit:** Screenshot of First Fragment + annotated tree diagram.

Lab 2 — Resources

1. Add a new color `violet` in `colors.xml` (`#8F00FF`).
2. Add a `Button` in `fragment_first.xml` using `android:backgroundTint="@color/violet"`.
3. Move hard-coded text `"GUI Components Demo"` into `strings.xml`.
4. **Submit:** Before/after screenshots + updated XML snippets.

Lab 3 — Fonts and Text

1. Add a fourth `TextView` with `android:textStyle="bold|italic"`.
2. Change title `textSize` from `24sp` to `28sp` and observe difference between `sp` and `dp`.
3. **Submit:** Short paragraph explaining sans vs serif vs monospace.

Lab 4 — GUI Components

1. Add a `Spinner` with three items (use `strings.xml` for entries).
2. Set `ProgressBar` progress to `80`.
3. **Submit:** Screenshot showing new widgets.

Lab 5 — Layouts

1. In `fragment_second.xml`, add an `ImageView` below the button using ConstraintLayout constraints.
2. Compare: why does First Fragment use `LinearLayout` and Second use `ConstraintLayout`?
3. **Submit:** Updated `fragment_second.xml` + 5-line comparison answer.

Lab 6 — Java and View Binding

1. In `FirstFragment.onViewCreated`, add a click listener on `button_red` that shows a `Toast` with text `"Red clicked"`.
2. Explain why `binding = null` is set in `onDestroyView`.
3. **Submit:** Modified `FirstFragment.java` + explanation.

Lab 7 — Navigation

1. Trace `R.id.action_FirstFragment_to_SecondFragment` from button click to destination in `nav_graph.xml`.
2. Add a `Toast` in `SecondFragment` when Previous is pressed (before navigate).
3. **Submit:** Flow diagram (hand-drawn) + code.

Lab 8 — MainActivity and Material

1. Change Snackbar message to your name and student ID.
2. Add a second menu item `About` that shows a `Toast`.
3. **Submit:** Screenshot of Snackbar and menu.

Capstone — Mini Extension (Choose One)

- **A:** Display `EditText` content in a `Snackbar` when a new button is pressed.
- **B:** Toggle night mode manually via menu item (research `AppCompatDelegate`).
- **C:** Add a third fragment to `nav_graph.xml` with a simple layout.

Capstone submit: Zip of project + 1-page report (aim, changes, screenshots).

---

11. Prerequisites Checklist

11.1 Knowledge Prerequisites

#	Topic	Required level
☐	Java OOP (classes, methods, interfaces)	Basic
☐	XML syntax (tags, attributes)	Basic
☐	Object-oriented event handling (listeners)	Basic
☐	HTTP / networking	Not required for this lab
☐	SQL / databases	Not required for this lab

11.2 Software Prerequisites

#	Item	Version / notes
☐	Android Studio	Latest stable recommended
☐	JDK	11+ (matches `compileOptions` in build.gradle)
☐	Android SDK	API 36 (compileSdk); install via SDK Manager
☐	Emulator or physical device	API 24 minimum
☐	Gradle sync successful	No red errors in Build window

11.3 Hardware Prerequisites

#	Item	Minimum
☐	RAM	8 GB (16 GB recommended)
☐	Free disk space	10 GB for SDK + emulator
☐	Virtualization	Enabled in BIOS for emulator (Intel VT-x / AMD-V)

11.4 Pre-Lab Verification (Student)

Run this checklist **before** Lab 1:

1. ☐ Open project `GUIComponentsFontsandColors` in Android Studio
2. ☐ **File → Sync Project with Gradle Files** completes without error
3. ☐ **Build → Make Project** succeeds
4. ☐ Run app — First Fragment appears with fonts and colored buttons
5. ☐ Tap **Go to Second Fragment** — navigation works
6. ☐ Tap **Previous** or Up — returns to First Fragment
7. ☐ Tap FAB — Snackbar appears

---

12. Troubleshooting

12.1 Build and Gradle

Problem	Likely cause	Solution
Gradle sync failed	No internet / proxy	Check network; configure proxy in Android Studio settings
`compileSdk` not found	SDK not installed	SDK Manager → install Android API 36 platform
View Binding errors (`cannot find symbol ActivityMainBinding`)	View Binding off or stale build	Enable `viewBinding true` in `app/build.gradle`; **Build → Clean Project → Rebuild**
JDK version mismatch	Wrong JDK selected	**File → Settings → Build → Gradle JDK** → select JDK 11+

12.2 Runtime Crashes

Problem	Likely cause	Solution
`InflateException` on layout	XML typo or missing library	Check Logcat line number in XML; verify Material dependency
App crashes on navigate	Missing action in nav_graph	Ensure `action_FirstFragment_to_SecondFragme` exists and ID matches Java

Problem	Likely cause	Solution
`NullPointerException` on binding	Accessing binding after `onDestroyView`	Only use binding between `onCreateView` and `onDestroyView`
FAB / Toolbar not visible	Theme or layout issue	Confirm `activity_main.xml` IDs match binding fields

12.3 UI and Resources

Problem	Likely cause	Solution
Color not applied to button	Wrong attribute	Use `android:backgroundTint` (Material), not deprecated `android:background` alone
String shows as `@string/...` on screen	Literal text in layout	Use `android:text="@string/name"` not `android:text="@string/name"` without quotes fix — ensure `@string` reference
Fonts look the same	Emulator font limitations	Test on physical device; verify `android:typeface` values
Second Fragment text not scrolling	Layout height	`NestedScrollView` should wrap content; check ConstraintLayout height

12.4 Emulator and Device

Problem	Likely cause	Solution
Emulator won’t start	HAXM / virtualization	Enable virtualization; use ARM image on Apple Silicon
App not installing	Insufficient storage	Free space on emulator; wipe data
minSdk error on old phone	Device API < 24	Use emulator API 24+ or newer device

12.5 Navigation-Specific

Problem	Likely cause	Solution
Up button does nothing	`onSupportNavigateUp` not wired	Verify `NavigationUI.setupActionBarWithNavContr` in `MainActivity`
Back stack confused	Multiple navigate calls	Use single action IDs; avoid duplicate `navigate()` without pop behavior
Wrong start screen	`startDestination` in nav_graph	Set `app:startDestination="@id/FirstFra`

12.6 Getting Help

1. Read **Logcat** (filter by package `com.example.guicomponentsfontsandcolors`).
2. Note the **file and line** from stack trace.
3. Compare your code with Sections 5–7 of this manual.
4. Ask instructor with: error message + screenshot + what you changed.

Document Information

Field	Value
Project	GUIComponentsFontsAndColors
Manual version	1.0
Sections	12 (as per lab syllabus)
Source alignment	Matches uploaded `app/src/main` sources

End of Lab Manual